Nexo

Arquitetura do banco de dados

Diagrama de relacionamentos, as oito tabelas com campos, tipos, chaves e restrições, os índices, a coleção vetorial, o arquivo de contas, as tabelas de integração, as consultas do painel de qualidade, as decisões de arquitetura e os caminhos para trocar de banco e escalar.

Versão 2.0.1
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Visão geral 4

1.1 Três lugares, três papéis 4
1.2 Criação automática 4
1.3 Convenções 4
1.4 Tabelas por área 5
1.5 O que mudou na 2.0 5

02 Diagrama de relacionamentos 6

2.1 Relacionamentos 6
2.2 Diagrama 6

03 Acervo 8

3.1 nx_document 8
3.2 Ciclo do status 8

04 Conversas e mensagens 10

4.1 nx_conversation 10
4.2 nx_conversation_document 10
4.3 nx_message 10
4.4 nx_message_source 11
4.5 nx_feedback 11

05 Integrações 13

5.1 nx_api_key 13
5.2 Por que só o hash da chave é guardado 14
5.3 nx_webhook 14
5.4 Como as entregas gravam last_status 15

06 Banco vetorial 16

6.1 Coleção 16
6.2 O que entra no embedding 16
6.3 Filtros e exclusão 16
6.4 Trocar o modelo de embeddings 17

07 Contas e arquivos 18

7.1 data/usuarios.json 18
7.2 data/uploads 19

Copyright (©) 2026 Erik Pimenta

7.3 data/.segredo 19

08 Consultas e índices 20

8.1 Consultas principais 20
8.2 Lista de índices 20
8.3 Consultas para o painel de qualidade 21

09 Decisões de arquitetura 23

9.1 Relacional e vetorial separados 23
9.2 Fontes gravadas com cópia do texto 23
9.3 Contas em arquivo 23
9.4 IDs gerados pela aplicação 23
9.5 Escopo por ausência 23
9.6 Chaves de API como hash 23
9.7 Webhooks sem fila persistente 24
9.8 Métricas calculadas na hora 24

10 Fluxos de dados 25

10.1 Envio de um PDF 25
10.2 Uma pergunta 25
10.3 Pergunta direta e busca 25
10.4 Avaliação 26
10.5 Exclusões 26

11 Outros bancos e escala 27

11.1 Outro banco relacional 27
11.2 Migrando dados do SQLite 27
11.3 Crescimento 27
11.4 Backup 28
11.5 DDL de referência 28

Copyright (©) 2026 Erik Pimenta

01

Visão geral

O Nexo guarda informações em três lugares, cada um com um papel. Este documento descreve a estrutura que os dados devem seguir, para quem administra o banco, migra para outro servidor ou integra o Nexo com outros sistemas.

1.1 Três lugares, três papéis

Camada	Onde fica (padrão)	O que guarda
Banco relacional	`data/nexo.db` (SQLite) ou o banco de `NEXO_DATABASE_URL`	Documentos, conversas, mensagens, fontes citadas, avaliações, chaves de API e webhooks
Banco vetorial	`data/chroma/` (ChromaDB) ou Qdrant (`data/qdrant/` ou servidor)	Trechos dos PDFs com os embeddings e os metadados de origem
Arquivos	`data/uploads/<id>.pdf`	PDFs originais, gravados com o id interno
Contas	`data/usuarios.json`	Usuários, perfis e hashes de senha
Segredo	`data/.segredo`	Chave das sessões quando `NEXO_SECRET` está vazio

A pasta `data` (ou a de `NEXO_DATA_DIR`) concentra tudo o que é local. Copiar essa pasta com o servidor parado é um backup completo.

1.2 Criação automática

As tabelas são criadas pelo SQLAlchemy na primeira conexão (`Base.metadata.create_all`), sem passo de migração. O `create_all` cria só as tabelas que faltam: ao atualizar da 1.5 para a 2.0, `nx_api_key` e `nx_webhook` aparecem na primeira inicialização e as seis tabelas antigas ficam como estão. No SQLite, cada conexão liga `PRAGMA foreign_keys=ON` , para que as regras ON DELETE funcionem, e `PRAGMA journal_mode=WAL` , para que leituras não esperem as gravações da fila de indexação. Os arquivos `docs/schema-postgresql.sql` e `docs/schema-sqlite.sql` trazem o DDL equivalente para quem prefere criar o banco manualmente.

1.3 Convenções

Convenção	Regra
Prefixo	Todas as tabelas começam com `nx_` , para conviver com outras aplicações no mesmo banco

Copyright (©) 2026 Erik Pimenta

Convenção	Regra
Chaves de texto	`id` de documentos, conversas, mensagens, chaves de API e webhooks é um UUID4 em hexadecimal, `varchar(32)`, gerado pela aplicação
Chaves numéricas	Fontes e avaliações usam integer autoincremento (SERIAL no PostgreSQL)
Datas	timestamp sem fuso, sempre em UTC; a interface converte para o horário local
Usuários	Colunas `owner`, `uploaded_by` e `created_by` guardam o username do arquivo de contas (não há tabela de usuários)
Estados	Textos em maiúsculas (PENDING, PROCESSING, READY, FAILED)
Listas curtas	Os eventos de um webhook ficam numa coluna de texto, separados por vírgula

1.4 Tabelas por área

Área	Tabelas
Acervo	`nx_document`
Conversas	`nx_conversation, nx_conversation_document`
Mensagens	`nx_message, nx_message_source`
Qualidade	`nx_feedback`
Integrações Novo	`nx_api_key, nx_webhook`

1.5 O que mudou na 2.0

- Duas tabelas novas: `nx_api_key` (chaves de API, só com o hash) e `nx_webhook` (destinos de avisos e resultado da última entrega).
- Nenhuma coluna mudou nas seis tabelas da 1.5. Bancos existentes continuam válidos sem migração manual.
- O painel de qualidade lê `nx_message`, `nx_message_source`, `nx_feedback` e `nx_conversation`; as consultas equivalentes estão na seção 8.3.
- A pergunta direta sem conversa (`POST /api/ask`) não grava nada no banco.

Copyright (©) 2026 Erik Pimenta

02

Diagrama de relacionamentos

Seis tabelas formam dois ramos que se encontram nas fontes citadas: o acervo de documentos e as conversas com suas mensagens. As duas tabelas de integração ficam à parte e se ligam às contas pelo nome de usuário.

2.1 Relacionamentos

Pai	Filho	Relação	Ao excluir o pai
`nx_conversation`	`nx_conversation_document`	1 para N	CASCADE: o escopo some com a conversa
`nx_document`	`nx_conversation_document`	1 para N	CASCADE: o documento sai do escopo de todas as conversas
`nx_conversation`	`nx_message`	1 para N	CASCADE: as mensagens somem com a conversa
`nx_message`	`nx_message_source`	1 para N	CASCADE
`nx_document`	`nx_message_source`	0..1 para N	SET NULL: o título e o trecho continuam no histórico
`nx_message`	`nx_feedback`	1 para N	CASCADE (uma avaliação por usuário)
conta (usuarios.json)	`nx_api_key`	1 para N	Sem chave estrangeira. A aplicação apaga as chaves junto com a conta
conta (usuarios.json)	`nx_webhook`	1 para N	Sem chave estrangeira. O webhook continua ativo; `created_by` é só registro

2.2 Diagrama

O diagrama mostra as chaves e as colunas usadas nas consultas principais; todas as colunas estão nos capítulos 3, 4 e 5.

Copyright (©) 2026 Erik Pimenta

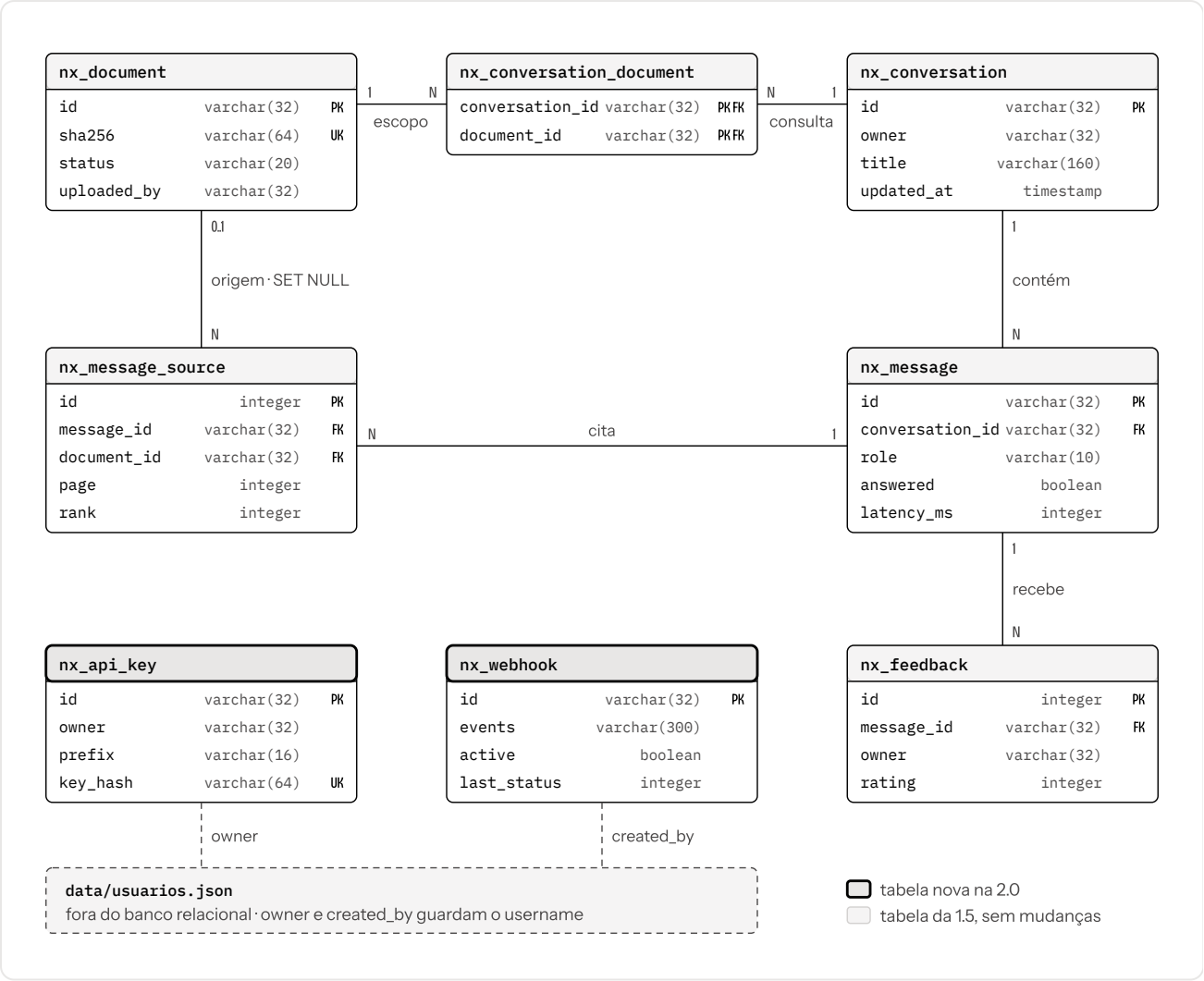


Figura 2.1 As oito tabelas do Nexo 2.0. "1" e "N" indicam a cardinalidade; as linhas tracejadas são referências lógicas ao arquivo de contas, sem chave estrangeira.

Copyright (©) 2026 Erik Pimenta

03

Acervo

O acervo é a lista de PDFs enviados. Cada linha acompanha o arquivo em data/uploads e os trechos no banco vetorial.

3.1 nx_document

Um registro por PDF enviado.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`varchar(32)`	não	UUID4 hex	PK. O PDF fica em `data/uploads/<id>.pdf`
`title`	`varchar(200)`	não	—	Título dos metadados do PDF ou o nome do arquivo; entra no texto do embedding
`filename`	`varchar(255)`	não	—	Nome original, saneado
`sha256`	`varchar(64)`	não	—	Único. Impede o envio do mesmo arquivo duas vezes
`size_bytes`	`integer`	não	—	Tamanho do arquivo, limitado por `NEXO_MAX_UPLOAD_MB`
`pages`	`integer`	não	0	Total de páginas do PDF
`chunks`	`integer`	não	0	Trechos gravados no banco vetorial
`status`	`varchar(20)`	não	PENDING	PENDING, PROCESSING, READY ou FAILED (indexado)
`error`	`text`	sim	—	Motivo da falha, quando houver
`uploaded_by`	`varchar(32)`	não	—	Usuário que enviou (indexado)
`created_at`	`timestamp`	não	agora (UTC)	Envio
`indexed_at`	`timestamp`	sim	—	Fim da indexação

3.2 Ciclo do status

Ao iniciar, o servidor recoloca na fila os documentos que ficaram em PENDING ou PROCESSING (queda no meio da indexação), então nenhum envio se perde. Ao chegar em READY ou FAILED, a fila dispara o webhook correspondente.

Copyright (©) 2026 Erik Pimenta

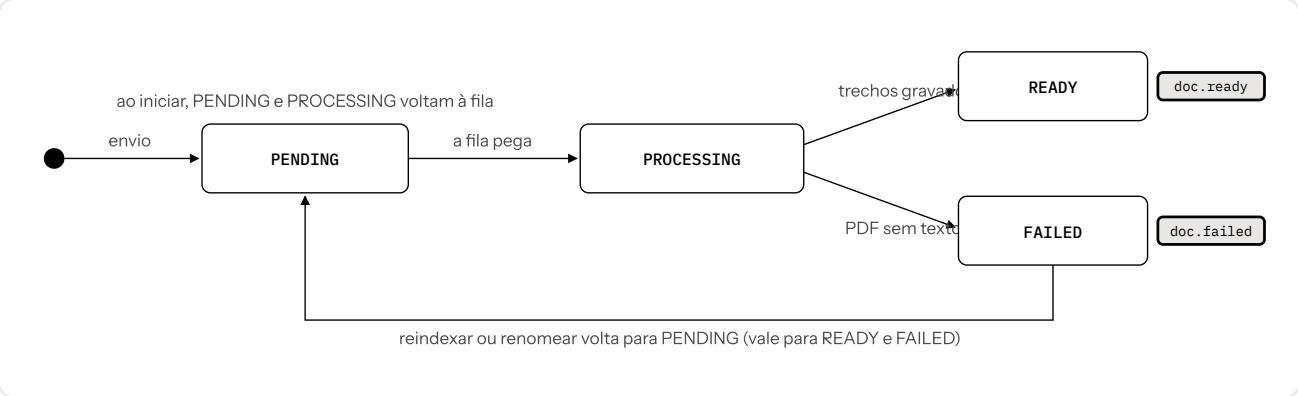


Figura 3.1 Estados de `nx_document.status` e os eventos de webhook disparados no fim da indexação.

Copyright (©) 2026 Erik Pimenta

04

Conversas e mensagens

Cada conversa pertence a um único usuário e guarda as perguntas, as respostas e os trechos que embasaram cada resposta.

4.1 nx_conversation

Uma conversa pertence a um único usuário (`owner`). Ninguém mais consegue abri-la, exportá-la ou usá-la em `/api/ask` .

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`varchar(32)`	não	UUID4 hex	PK
`owner`	`varchar(32)`	não	—	Usuário dono (indexado)
`title`	`varchar(160)`	não	"Nova conversa"	Recebe a primeira pergunta; pode ser renomeado; dá nome ao arquivo exportado
`created_at`	`timestamp`	não	agora (UTC)	Criação
`updated_at`	`timestamp`	não	agora (UTC)	Atualizado a cada pergunta ou mudança; indexado para ordenar a lista

4.2 nx_conversation_document

Escopo da conversa. Sem linhas, a conversa consulta todos os documentos prontos. Com linhas, a busca vetorial é filtrada pelos documentos listados.

Coluna	Tipo	Nulo	Regra
`conversation_id`	`varchar(32)`	não	PK e FK para `nx_conversation` , ON DELETE CASCADE
`document_id`	`varchar(32)`	não	PK e FK para `nx_document` , ON DELETE CASCADE

4.3 nx_message

Perguntas (`role = user`) e respostas (`role = assistant`). O painel de qualidade conta cada mensagem `user` como uma pergunta e olha a mensagem `assistant` seguinte para saber se foi respondida.

Coluna	Tipo	Nulo	Regra
`id`	`varchar(32)`	não	PK

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Regra
`conversation_id`	`varchar(32)`	não	FK para `nx_conversation`, ON DELETE CASCADE (indexado)
`role`	`varchar(10)`	não	`user` ou `assistant`
`content`	`text`	não	Texto da pergunta ou da resposta, com as citações [n]
`answered`	`boolean`	sim	`false` quando nada relevante foi encontrado; base da taxa de resposta e do evento `question.unanswered`
`grounded`	`boolean`	sim	Resultado da verificação de fidelidade (`NEXO_VERIFY_ANSWERS=true`); nulo se desligada
`model`	`varchar(120)`	sim	Modelo que respondeu (por exemplo llama3.1 ou extrativo)
`latency_ms`	`integer`	sim	Tempo gasto na resposta; base da latência média e do p95 do painel
`created_at`	`timestamp`	não	Índice composto com `conversation_id`

4.4 nx_message_source

Os trechos efetivamente enviados ao modelo, na ordem das citações [1], [2]...

Coluna	Tipo	Nulo	Regra
`id`	`integer`	não	PK autoincremento
`message_id`	`varchar(32)`	não	FK para `nx_message`, ON DELETE CASCADE (indexado)
`document_id`	`varchar(32)`	sim	FK para `nx_document`, ON DELETE SET NULL (indexado)
`document_title`	`varchar(200)`	não	Cópia do título; mantém o histórico legível depois de exclusões
`chunk_id`	`varchar(80)`	não	Id do trecho no banco vetorial
`page`	`integer`	não	Página do PDF, a partir de 1
`rank`	`integer`	não	Número da citação [n] na resposta
`similarity`	`float`	sim	Similaridade de cosseno da busca vetorial
`rerank_score`	`float`	sim	Nota do reranker; nula com `NEXO_RERANKER=none`
`excerpt`	`text`	não	Texto do trecho (até 1.200 caracteres)

4.5 nx_feedback

Avaliação de cada resposta. Um usuário tem no máximo uma avaliação por resposta. Quando a avaliação passa a ser −1, o Nexo dispara o evento `feedback.negative`.

Coluna	Tipo	Nulo	Regra
`id`	`integer`	não	PK autoincremento
`message_id`	`varchar(32)`	não	FK para `nx_message`, ON DELETE CASCADE
`owner`	`varchar(32)`	não	Usuário que avaliou; único junto com `message_id` (`uk_feedback_message_owner`)
`rating`	`integer`	não	1 (útil) ou −1 (não ajudou). Enviar 0 pela API remove a avaliação

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Regra
`comment`	`text`	sim	Comentário opcional (via API)
`created_at`	`timestamp`	não	Data da avaliação

Copyright (©) 2026 Erik Pimenta

05

Integrações Novo

Duas tabelas novas sustentam as integrações da 2.0: as chaves de API, com que outros sistemas se autenticam, e os webhooks, que avisam outros sistemas sobre eventos.

5.1 nx_api_key

Uma linha por chave criada. Cada usuário vê e revoga só as próprias chaves.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`varchar(32)`	não	UUID4 hex	PK. Usado em `DELETE /api/keys/{id}`
`owner`	`varchar(32)`	não	—	Username do dono (indexado). A chave age com as permissões dele
`name`	`varchar(80)`	não	—	Nome dado pelo usuário, de 1 a 80 caracteres
`prefix`	`varchar(16)`	não	—	Os 12 primeiros caracteres da chave (`nxk_` + 8), para reconhecê-la nas telas
`key_hash`	`varchar(64)`	não	—	Único. SHA-256 da chave completa, em hexadecimal
`created_at`	`timestamp`	não	agora (UTC)	Criação
`last_used_at`	`timestamp`	sim	—	Atualizado a cada pedido autenticado com a chave; nulo até o primeiro uso

Restrições e índices

- `PRIMARY KEY (id)`.
- `UNIQUE (key_hash)`: cada pedido localiza a chave por esse valor, em uma única leitura pelo índice da restrição.
- `ix_nx_api_key_owner` em `owner`: lista as chaves do usuário e a contagem de `integrations.api_keys` em `/api/system/status`.

Copyright (©) 2026 Erik Pimenta

5.2 Por que só o hash da chave é guardado

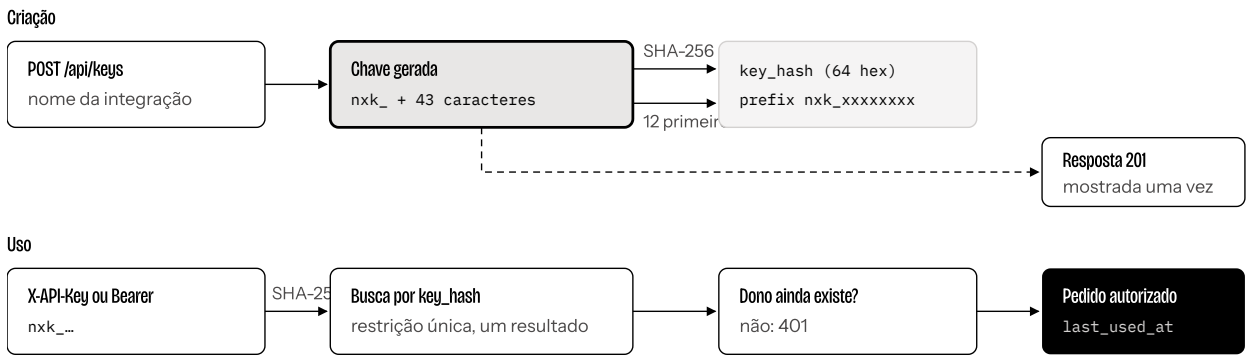


Figura 5.1 A chave completa só existe na resposta da criação. Daí em diante, o Nexo compara hashes.

- **Vazamento do banco não vira acesso.** Uma cópia do banco ou de um backup traz só hashes; com eles não se monta um pedido válido.
- **SHA-256 basta aqui.** Diferente de uma senha, a chave é aleatória e longa (43 caracteres depois de `nxk_`), então não há como adivinhá-la por dicionário. Um hash rápido e sem sal permite buscar a chave direto pela restrição única, sem percorrer a tabela. Senhas continuam em bcrypt.
- **Mostrada uma única vez.** Como o banco não tem a chave, não existe "ver de novo". Perdeu, revoga e cria outra.
- **O prefixo identifica sem expor.** Os 12 primeiros caracteres aparecem nas listas e ajudam a saber qual chave está num sistema, sem permitir o uso.
- **Conta excluída, chave apagada.** Excluir uma conta apaga as chaves dela. Mesmo assim, na autenticação, depois de achar o hash, o Nexo confere se o `owner` ainda existe em `usuarios.json`; se não existe, o pedido recebe 401.

5.3 nx_webhook

Uma linha por destino cadastrado. Só administradores criam, alteram e excluem webhooks.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`varchar(32)`	não	UUID4 hex	PK
`url`	`varchar(500)`	não	—	Destino `http` ou `https`, validado na criação (400 `INVALID_URL`)
`events`	`varchar(300)`	não	—	Subconjunto não vazio de `document.ready`, `document.failed`, `question.unanswered`, `feedback.negative`, separado por vírgula (400 `INVALID_EVENTS` se vazio ou desconhecido)
`secret`	`varchar(80)`	não	gerado	Segredo do HMAC. Sem valor na criação, `secrets.token_urlsafe(24)`
`active`	`boolean`	não	true	Pausado (`false`) não recebe eventos; muda por `PATCH`
`created_by`	`varchar(32)`	não	—	Administrador que cadastrou
`created_at`	`timestamp`	não	agora (UTC)	Criação
`last_status`	`integer`	sim	—	Status HTTP da última entrega; nulo se não houve resposta

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
`last_error`	`text`	sim	—	Mensagem de erro da última entrega; nula quando deu certo
`last_delivery_at`	`timestamp`	sim	—	Quando a última entrega terminou

Sem índices além da chave primária: a tabela tem poucas linhas e é lida inteira a cada evento.

5.4 Como as entregas gravam last_status

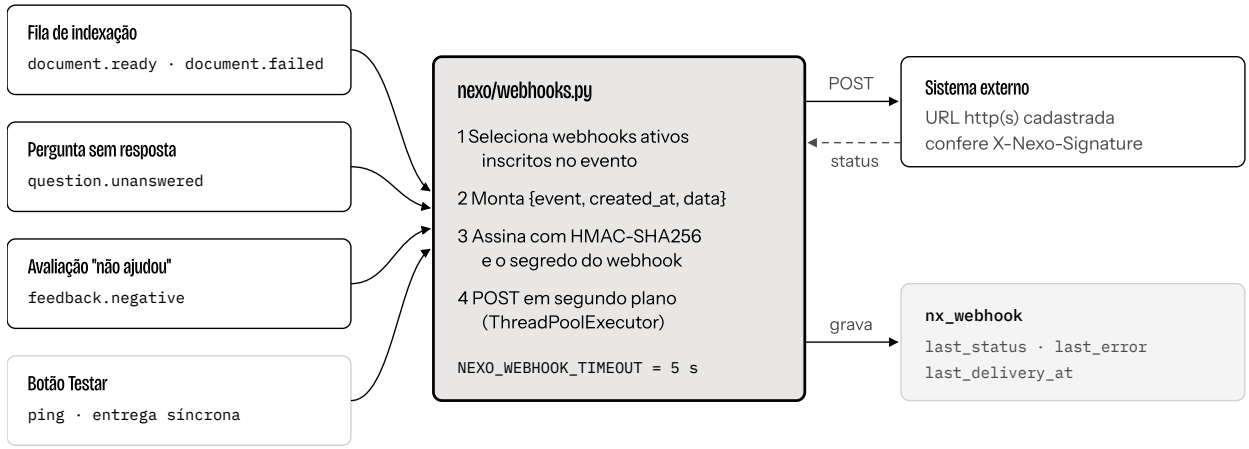


Figura 5.2 Os quatro eventos e o teste passam pelo mesmo módulo. O resultado de cada entrega é gravado na linha do webhook.

Cada entrega roda numa thread do `ThreadPoolExecutor` (o teste roda na hora, dentro do pedido), usa `urllib.request` com tempo limite de `NEXO_WEBHOOK_TIMEOUT` segundos e, ao terminar, abre uma sessão própria no banco e atualiza as três colunas do webhook:

O que aconteceu	last_status	last_error	last_delivery_at
Receptor respondeu 2xx	o status (200, 204…)	nulo	agora (UTC)
Receptor respondeu 3xx, 4xx ou 5xx	o status (401, 500…)	descrição do erro HTTP	agora (UTC)
Tempo esgotado, DNS ou conexão recusada	nulo	a mensagem da falha	agora (UTC)

- As colunas guardam só a **última** entrega. Não há histórico de entregas nem nova tentativa automática.
- Entregas simultâneas para o mesmo webhook gravam na ordem em que terminam; vale a última.
- A falha de uma entrega nunca afeta a operação que a disparou: a pergunta, a avaliação ou a indexação seguem normalmente.

Copyright (©) 2026 Erik Pimenta

06

Banco vetorial

Os trechos dos PDFs ficam numa coleção do ChromaDB (padrão) ou do Qdrant, com os embeddings e os metadados que ligam cada trecho ao documento de origem.

6.1 Coleção

Coleção padrão: `nexo_trechos` (`NEXO_COLLECTION`), com distância de cosseno.

Campo	Conteúdo
`id`	UUID v5 calculado de `nexo:<id do documento>:<posição do trecho>` ; reindexar gera os mesmos ids
`embedding`	Vetor do modelo configurado em `NEXO_EMBED_MODEL`
`document`	Texto do trecho (até `NEXO_CHUNK_SIZE` tokens, com `NEXO_CHUNK_OVERLAP` de sobreposição)
`metadata.source_id`	`nx_document.id` do PDF de origem
`metadata.title`	Título do documento
`metadata.filename`	Nome do arquivo
`metadata.page`	Página do PDF, começando em 1
`metadata.chunk_index`	Posição do trecho dentro do documento

6.2 O que entra no embedding

Só o título do documento entra no texto usado para gerar o embedding, junto com o conteúdo do trecho. Isso ajuda perguntas como "no manual da bomba, ...". `source_id` , `filename` , `page` e `chunk_index` ficam fora do embedding e servem só para filtrar e citar.

6.3 Filtros e exclusão

Operação	Como é feita
Escopo da conversa	Filtro de metadados `source_id` igual ao documento (um) ou dentro da lista (vários)
Busca nos trechos e /api/ask	Mesmo filtro, a partir de `document_ids` do pedido; lista vazia consulta tudo
Excluir documento	Apaga todos os vetores com `source_id` do documento antes de apagar a linha e o PDF

Copyright (©) 2026 Erik Pimenta

Operação	Como é feita
Reindexar	Apaga os vetores do documento e grava os novos; nunca há duplicação

6.4 Trocar o modelo de embeddings

Vetores de modelos diferentes não são comparáveis. Depois de mudar `NEXO_EMBED_PROVIDER` ou `NEXO_EMBED_MODEL`, use uma coleção nova (`NEXO_COLLECTION`) ou reindexe todos os documentos pela tela Documentos.

Copyright (©) 2026 Erik Pimenta

07

Contas e arquivos

As contas ficam num arquivo JSON, fora do banco relacional, para que o login funcione mesmo antes de qualquer banco externo ser configurado.

7.1 data/usuarios.json

```
{
  "versao": 1,
  "usuarios": [
    {
      "username": "admin",
      "name": "Administrador",
      "role": "admin",
      "password_hash": "$2b$12$...",
      "created_at": "2026-01-01T12:00:00+00:00",
      "last_login_at": null
    }
  ]
}
```

Campo	Regra
`username`	3 a 32 caracteres minúsculos: letras, números, ponto, hífen e sublinhado. Referenciado por `owner`, `uploaded_by` e `created_by`
`name`	Nome exibido na interface
`role`	`admin` ou `user`. Muda por `PATCH /api/users/{username}`; o último admin não pode ser rebaixado
`password_hash`	Hash bcrypt; a senha nunca é guardada
`created_at / last_login_at`	ISO 8601 em UTC

- O arquivo é regravado de forma atômica: a versão nova vai para um arquivo temporário na mesma pasta e substitui o antigo de uma vez. Uma trava em memória impede duas gravações ao mesmo tempo dentro do processo.
- Se não houver nenhum administrador (por exemplo, na primeira execução), o Nexo cria o login pré-definido `admin` / `Nexo@2026`.
- Excluir uma conta (`DELETE /api/users/{username}`) remove a entrada do arquivo e apaga do banco as chaves de API e as conversas dela (com mensagens, fontes e avaliações, em cascata).
- Para redefinir uma senha esquecida: `python scripts/redefinir_senha.py nome-do-usuario`.

Copyright (©) 2026 Erik Pimenta

7.2 data/uploads

Cada PDF é gravado como `<id>.pdf`, com o id de `nx_document`. O nome original fica só na coluna `filename`, o que evita conflitos e nomes perigosos no disco.

7.3 data/.segredo

Chave aleatória criada quando `NEXO_SECRET` está vazio. Apagar o arquivo invalida todas as sessões abertas. Chaves de API não dependem dele: continuam válidas porque são conferidas pelo hash.

Copyright (©) 2026 Erik Pimenta

08

Consultas e índices

Cada índice existe por causa de uma consulta da aplicação. O painel de qualidade é calculado em Python a partir de consultas simples; as versões em SQL abaixo servem para relatórios e conferência.

8.1 Consultas principais

Tela ou rotina	Consulta	Índice usado
Lista de conversas	`nx_conversation` por `owner`, ordenada por `updated_at` desc	`ix_nx_conversation_owner`, `ix_nx_conversation_updated_at`
Abrir ou exportar conversa	`nx_message` por `conversation_id`, ordenada por `created_at`	`ix_nx_message_conversation_created`
Fontes de uma resposta	`nx_message_source` por `message_id`, ordenada por `rank`	`ix_nx_message_source_message_id`
Excluir documento	`nx_message_source` por `document_id` (SET NULL)	`ix_nx_message_source_document_id`
Envio de PDF	`nx_document` por `sha256`	restrição única de `sha256`
Retomar a fila	`nx_document` com status em PENDING ou PROCESSING	`ix_nx_document_status`
Sistema e conta	contagem de `nx_document` por status e soma de `chunks`; chaves do usuário; total de webhooks	`ix_nx_document_status`, `ix_nx_api_key_owner`
Autenticar chave de API	`nx_api_key` por `key_hash`	restrição única de `key_hash`
Integrações	`nx_api_key` por `owner`; `nx_webhook` inteira	`ix_nx_api_key_owner`
Disparo de evento	`nx_webhook` com `active` verdadeiro, filtrada pelo evento em Python	leitura completa (tabela pequena)
Painel de qualidade	`nx_message` do período, unida a `nx_conversation` para o escopo	`ix_nx_conversation_owner`, `ix_nx_message_conversation_created`

8.2 Lista de índices

Índice	Tabela	Colunas
`ix_nx_document_status`	`nx_document`	`status`
`ix_nx_document_uploaded_by`	`nx_document`	`uploaded_by`
`ix_nx_conversation_owner`	`nx_conversation`	`owner`

Copyright (©) 2026 Erik Pimenta

Índice	Tabela	Colunas
ix_nx_conversation_updated_at	nx_conversation	updated_at
ix_nx_message_conversation_id	nx_message	conversation_id
ix_nx_message_conversation_created	nx_message	conversation_id, created_at
ix_nx_message_source_message_id	nx_message_source	message_id
ix_nx_message_source_document_id	nx_message_source	document_id
uk_feedback_message_owner	nx_feedback	message_id, owner (único)
único de `sha256`	nx_document	sha256
ix_nx_api_key_owner Novo	nx_api_key	owner
único de `key_hash` Novo	nx_api_key	key_hash

8.3 Consultas para o painel de qualidade

As consultas abaixo reproduzem os números de `GET /api/metrics` em SQL padrão (testadas no SQLite e no PostgreSQL). Para o escopo de um usuário, acrescente `AND c.owner = 'nome'`. Troque `:inicio` pela data inicial do período, em UTC.

Perguntas, respondidas e taxa de resposta

```
SELECT COUNT(*)                                          AS respostas,
       SUM(CASE WHEN m.answered THEN 1 ELSE 0 END)        AS respondidas,
       SUM(CASE WHEN NOT m.answered THEN 1 ELSE 0 END)    AS sem_resposta,
       ROUND(AVG(CASE WHEN m.answered THEN 1.0 ELSE 0 END), 3) AS taxa
FROM nx_message m
JOIN nx_conversation c ON c.id = m.conversation_id
WHERE m.role = 'assistant' AND m.created_at >= :inicio;
```

Perguntas e perguntas sem resposta por dia

```
SELECT DATE(m.created_at) AS dia,
       COUNT(*) AS perguntas,
       SUM(CASE WHEN m.answered = FALSE THEN 1 ELSE 0 END) AS sem_resposta
FROM nx_message m
JOIN nx_conversation c ON c.id = m.conversation_id
WHERE m.role = 'assistant' AND m.created_at >= :inicio
GROUP BY DATE(m.created_at)
ORDER BY dia;
```

O painel conta perguntas pelas mensagens `user` e completa com zero os dias sem atividade; contar pelas respostas, como acima, dá o mesmo número quando toda pergunta foi respondida ou recusada.

Copyright (©) 2026 Erik Pimenta

Lacunas do acervo: perguntas sem resposta mais recentes

```
SELECT q.content AS pergunta, q.created_at AS feita_em, q.conversation_id
FROM nx_message a
JOIN nx_message q ON q.conversation_id = a.conversation_id
                 AND q.role = 'user'
                 AND q.created_at = (SELECT MAX(x.created_at) FROM nx_message x
                                     WHERE x.conversation_id = a.conversation_id
                                       AND x.role = 'user' AND x.created_at <= a.created_at)
WHERE a.role = 'assistant' AND a.answered = FALSE AND a.created_at >= :inicio
ORDER BY q.created_at DESC
LIMIT 10;
```

Respostas avaliadas como "não ajudou"

```
SELECT f.created_at, f.owner, m.id AS message_id, m.conversation_id, m.content AS resposta
FROM nx_feedback f
JOIN nx_message m ON m.id = f.message_id
WHERE f.rating = -1 AND f.created_at >= :inicio
ORDER BY f.created_at DESC
LIMIT 10;
```

Documentos mais citados

```
SELECT s.document_id, MAX(s.document_title) AS titulo, COUNT(*) AS citacoes
FROM nx_message_source s
JOIN nx_message m ON m.id = s.message_id
WHERE m.created_at >= :inicio
GROUP BY s.document_id
ORDER BY citacoes DESC
LIMIT 6;
```

Latência média

```
SELECT ROUND(AVG(latency_ms)) AS media_ms
FROM nx_message
WHERE role = 'assistant' AND latency_ms IS NOT NULL AND created_at >= :inicio;
```

Por que o p95 fica em Python. Percentis têm sintaxe diferente em cada banco (`percentile_cont` no PostgreSQL, nenhuma função no SQLite). O Nexo lê as latências do período e calcula o p95 na aplicação, o que funciona igual nos quatro bancos suportados. Documentos excluídos aparecem com `document_id` nulo e o título copiado.

Copyright (©) 2026 Erik Pimenta

09

Decisões de arquitetura

Por que o banco é como é. Cada decisão tem um custo, e ele está escrito ao lado.

9.1 Relacional e vetorial separados

O banco relacional guarda o que precisa de integridade e histórico (documentos, conversas, citações, integrações); o vetorial guarda só o que serve à busca. A ligação entre os dois é o `source_id` nos metadados e o `chunk_id` nas fontes. Assim, dá para trocar ChromaDB por Qdrant sem mexer nas tabelas.

9.2 Fontes gravadas com cópia do texto

`nx_message_source` guarda o título e o trecho, não só a referência. Uma resposta antiga continua auditável, e exportável, mesmo depois que o documento é excluído ou reindexado com outro tamanho de trecho.

9.3 Contas em arquivo

Sem tabela de usuários, o primeiro acesso funciona antes de qualquer banco externo e a redefinição de senha é um script de uma linha. O preço é que `owner`, `uploaded_by` e `created_by` não têm chave estrangeira: a aplicação confere o usuário a cada pedido, inclusive nos pedidos com chave de API.

9.4 IDs gerados pela aplicação

UUIDs em hexadecimal permitem gravar o PDF em disco antes do commit e evitam colisões ao juntar bancos. As tabelas de alto volume e sem exposição externa (`nx_message_source`, `nx_feedback`) usam autoincremento, que ocupa menos espaço.

9.5 Escopo por ausência

Uma conversa sem linhas em `nx_conversation_document` consulta tudo o que está pronto. Documentos novos entram automaticamente nas conversas abertas, sem atualizar escopos.

9.6 Chaves de API como hash

Guardar só o SHA-256 torna inofensivo um vazamento do banco ou de um backup. O custo é não poder mostrar a chave de novo; o prefixo de 12 caracteres compensa na hora de identificar qual chave está em qual sistema.

Copyright (©) 2026 Erik Pimenta

9.7 Webhooks sem fila persistente

As entregas saem de um pool de threads em memória e o resultado fica na própria linha do webhook. É simples e não exige outro serviço (Redis, fila de mensagens). O custo: um evento em trânsito se perde se o servidor parar naquele instante, e não há histórico de entregas. Para quem precisa de garantia, o receptor pode conferir o estado pela API (por exemplo, listar documentos) ao reiniciar.

9.8 Métricas calculadas na hora

O painel não tem tabela de agregados: cada consulta lê as mensagens do período e calcula em Python. Os números estão sempre certos e não há rotina para manter. Com muitos milhões de mensagens, períodos longos ficam mais lentos; aí vale passar para PostgreSQL e limitar o período.

Copyright (©) 2026 Erik Pimenta

10

Fluxos de dados

O que cada operação grava, na ordem em que grava.

10.1 Envio de um PDF

1. A API confere o cabeçalho `%PDF`, o tamanho e o SHA-256 (duplicado devolve 409 `DUPLICATE`).
2. Grava `data/uploads/<id>.pdf` e a linha em `nx_document` com PENDING.
3. A fila muda para PROCESSING, extrai o texto por página, divide em trechos e grava os vetores.
4. Termina em READY com `pages`, `chunks` e `indexed_at`, ou em FAILED com `error`, e dispara `document.ready` ou `document.failed`.

10.2 Uma pergunta

1. Grava a pergunta em `nx_message` (`role = user`).
2. Reescreve a pergunta com as últimas `NEXO_HISTORY_TURNS` trocas, busca no banco vetorial com o filtro de escopo, aplica os cortes e o reranking.
3. Grava a resposta (`role = assistant`) com `answered`, `grounded`, `model` e `latency_ms`, e uma linha em `nx_message_source` por trecho citado.
4. Atualiza `nx_conversation.updated_at` e, na primeira pergunta, o título.
5. Se `answered` é falso, dispara `question.unanswered`.

10.3 Pergunta direta e busca

Chamada	O que é gravado
`POST /api/ask` sem `conversation_id`	Nenhuma mensagem. Só `last_used_at` da chave, se o pedido veio com chave; sem resposta, dispara `question.unanswered` com `conversation_id` nulo
`POST /api/ask` com `conversation_id`	O mesmo que uma pergunta no chat (seção 10.2)
`POST /api/search`	Nada. A busca só lê o banco vetorial
`GET /api/conversations/{id}/export`	Nada. Lê conversa, mensagens e fontes

Copyright (©) 2026 Erik Pimenta

10.4 Avaliação

Grava ou atualiza a linha de `nx_feedback` do usuário para a resposta (rating 0 apaga). Quando a avaliação passa a ser −1, dispara `feedback.negative` com a pergunta e a resposta; repetir −1 não dispara de novo.

10.5 Exclusões

O que é excluído	Efeito
Documento	Vetores apagados, PDF removido, linha apagada, escopos apagados (CASCADE), fontes com `document_id` nulo (SET NULL)
Conversa	Escopo, mensagens, fontes e avaliações apagados (CASCADE)
Avaliação (rating 0)	Só a linha de `nx_feedback`
Chave de API	Só a linha de `nx_api_key` ; pedidos com ela recebem 401 na hora
Webhook	Só a linha de `nx_webhook` ; entregas já em andamento terminam, mas o resultado não tem mais onde ser gravado
Conta	Entrada de `usuarios.json` , linhas de `nx_api_key` do dono e as conversas dele, com mensagens, fontes e avaliações (CASCADE)

Copyright (©) 2026 Erik Pimenta

11

Outros bancos e escala

Como trocar o SQLite por um servidor de banco, migrar os dados e acompanhar o crescimento.

11.1 Outro banco relacional

Defina `NEXO_DATABASE_URL` no `.env` e instale o driver:

Banco	Driver	Exemplo de URL
PostgreSQL	`pip install "psycopg[binary]"`	`postgresql+psycopg://nexo:senha@localhost:5432/nexo`
MySQL / MariaDB	`pip install pymysql`	`mysql+pymysql://nexo:senha@localhost:3306/nexo`
SQL Server	`pip install pyodbc`	`mssql+pyodbc://nexo:senha@servidor/nexo?driver=ODBC+Driver+18+for+SQL+Server`

O `docker-compose.yml` tem um serviço PostgreSQL de exemplo (`docker compose --profile postgres up -d`). Troque a senha do arquivo antes de usar.

11.2 Migrando dados do SQLite

1. Pare o servidor e faça uma cópia de `data/nexo.db` .
2. Crie o banco novo com `docs/schema-postgresql.sql` (ou deixe o Nexo criar ao iniciar com a URL nova).
3. Copie as tabelas na ordem das chaves: `nx_document` , `nx_conversation` , `nx_conversation_document` , `nx_message` , `nx_message_source` , `nx_feedback` , `nx_api_key` , `nx_webhook` . As duas últimas não dependem de nenhuma outra.
4. No PostgreSQL, ajuste as sequências de `nx_message_source.id` e `nx_feedback.id` para o maior id copiado.
5. Defina `NEXO_DATABASE_URL` e suba o servidor. O banco vetorial e os PDFs não mudam, e as chaves de API continuam valendo, porque o hash vai junto.

11.3 Crescimento

Tabela ou coleção	Cresce com	O que fazer quando ficar grande
Coleção vetorial	Páginas indexadas	Passar para Qdrant em servidor (`NEXO_QDRANT_URL`)

Copyright (©) 2026 Erik Pimenta

Tabela ou coleção	Cresce com	O que fazer quando ficar grande
`nx_message` e `nx_message_source`	Perguntas feitas	PostgreSQL; apagar conversas antigas apaga as fontes em cascata. Integrações que usam `/api/ask` sem conversa não fazem a tabela crescer
`nx_document`	PDFs enviados	Cresce pouco; o peso está em `data/uploads`
`nx_api_key` e `nx_webhook`	Integrações cadastradas	Ficam pequenas; revogue chaves sem uso olhando `last_used_at`

11.4 Backup

Pare o servidor e copie a pasta `data` inteira: banco SQLite, vetores locais, PDFs, usuários e segredo. Com bancos externos (PostgreSQL, Qdrant, ChromaDB em servidor), faça também o backup deles pelas ferramentas próprias, no mesmo momento, para que as referências entre eles continuem batendo. O backup contém os segredos dos webhooks: guarde-o com o mesmo cuidado de uma senha.

11.5 DDL de referência

O DDL completo está em `docs/schema-postgresql.sql` e `docs/schema-sqlite.sql`. Trecho do PostgreSQL com uma tabela antiga e as duas novas:

```
CREATE TABLE nx_message_source (
    id SERIAL NOT NULL,
    message_id VARCHAR(32) NOT NULL,
    document_id VARCHAR(32),
    document_title VARCHAR(200) NOT NULL,
    chunk_id VARCHAR(80) NOT NULL,
    page INTEGER NOT NULL,
    rank INTEGER NOT NULL,
    similarity FLOAT,
    rerank_score FLOAT,
    excerpt TEXT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(message_id) REFERENCES nx_message (id) ON DELETE CASCADE,
    FOREIGN KEY(document_id) REFERENCES nx_document (id) ON DELETE SET NULL
);
```

Copyright (©) 2026 Erik Pimenta

```
CREATE TABLE nx_api_key (
    id VARCHAR(32) NOT NULL,
    owner VARCHAR(32) NOT NULL,
    name VARCHAR(80) NOT NULL,
    prefix VARCHAR(16) NOT NULL,
    key_hash VARCHAR(64) NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    last_used_at TIMESTAMP WITHOUT TIME ZONE,
    PRIMARY KEY (id),
    UNIQUE (key_hash)
);
CREATE INDEX ix_nx_api_key_owner ON nx_api_key (owner);

CREATE TABLE nx_webhook (
    id VARCHAR(32) NOT NULL,
    url VARCHAR(500) NOT NULL,
    events VARCHAR(300) NOT NULL,
    secret VARCHAR(80) NOT NULL,
    active BOOLEAN NOT NULL,
    created_by VARCHAR(32) NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    last_status INTEGER,
    last_error TEXT,
    last_delivery_at TIMESTAMP WITHOUT TIME ZONE,
    PRIMARY KEY (id)
);
```

Copyright (©) 2026 Erik Pimenta